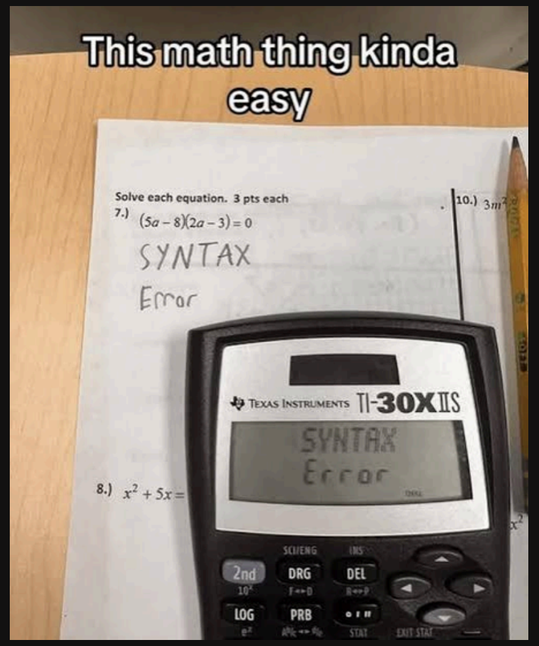
Semana 11 - martes

APUNTES 13/10/26

Lun Valverde Villalobos



ANÁLISIS SINTÁCTICO



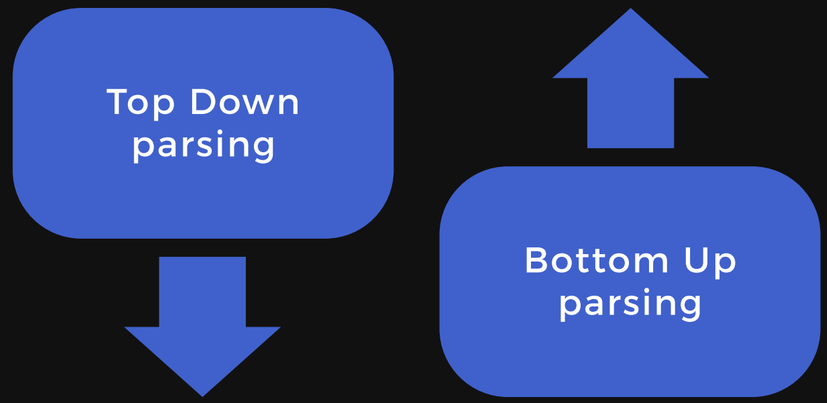
Análisis Sintáctico

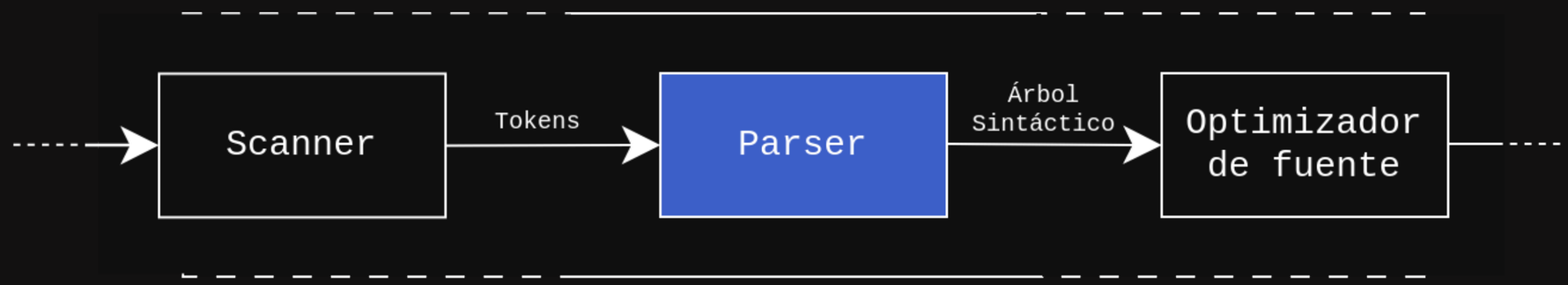
Hace mucho, mucho tiempo habíamos hablado sobre el análisis sintáctico...

- Conocido como parser.
- Revisa que la sintaxis de un programa esté correcta.
- Toma una secuencia de tokens y valida que correspondan a construcciones válidas.
- Requiere de una gramática. [1]

Parsing

- Existen muchos algoritmos de parsing.
- Hay 2 tipos: Top Down y Bottom Up.







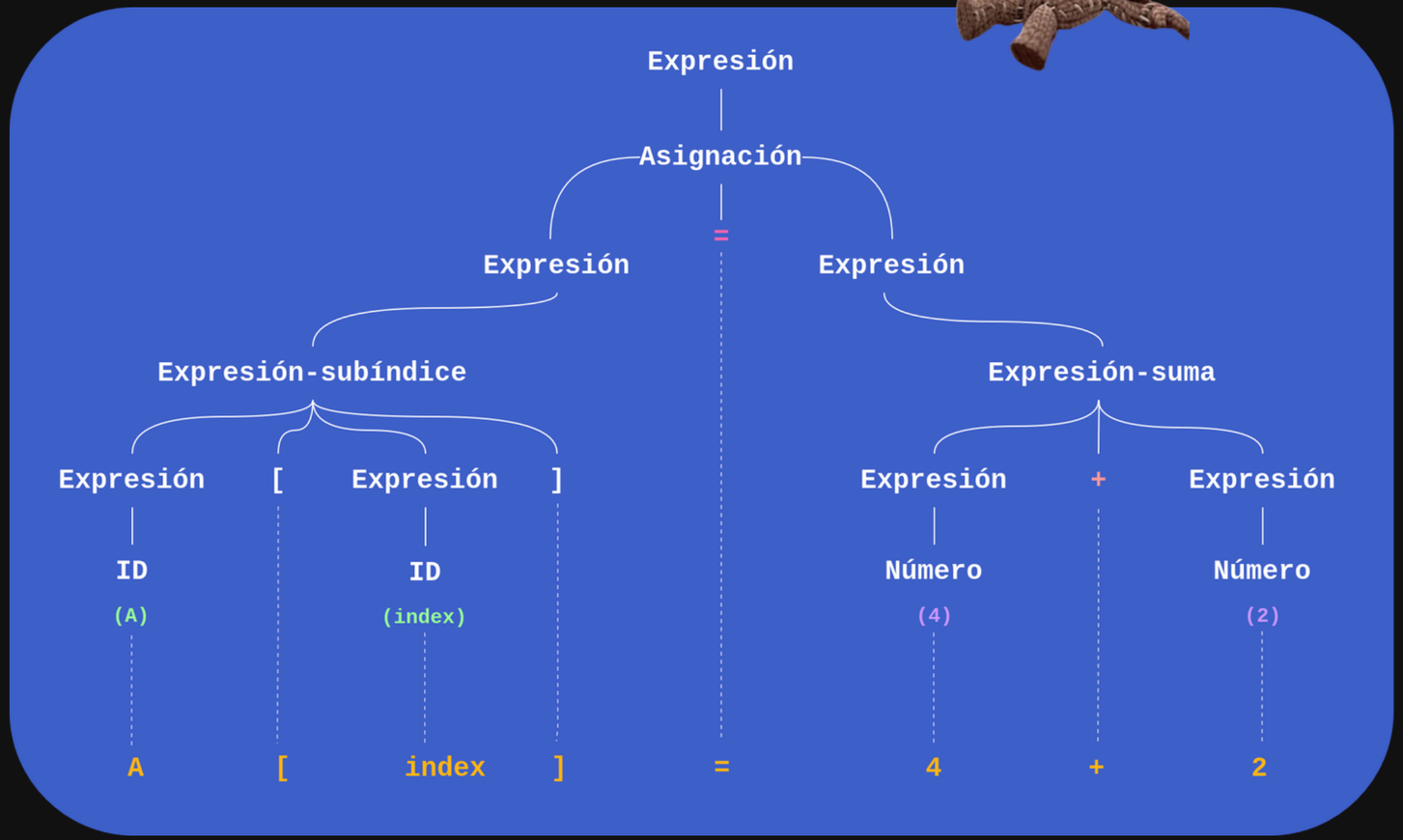
[1] Una gramática es un conjunto de reglas de producción que determina cómo se construyen las cadenas en un lenguaje.

Árbol sintáctico

¿Cómo se vería el árbol sintáctico de **A[index] = 4 + 2**?









Varias de las imágenes que hice las pueden ver también aquí.

Errores de Sintaxis

- Aunque el manejo de errores es relativamente fácil en el análisis léxico (scanner), en el análisis sintáctico es muy difícil.
- El parser debe realizar varias cosas cuando encuentra un error:
  - **R**eportar (report).
  - **R**ecuperar (recover).
  - **R**eparar (repair).

Olvídense de Reducir, Reutilizar y Reciclar, estas son las nuevas 3 R's! [2]





```
lun@lun-linux:~/Documents$ python3 script.py
  File "/home/lun/Documents/script.py", line ???
    this line does NOT have the error
                    ^

SyntaxError: fuck you
```

[2] MENTIRA!, cuiden el medio ambiente por favor.

Reportar

- Con los algoritmos de parsing, encontrar un error es automático.
- El problema es que, aunque sepamos que hay un error, no sabemos aún *dónde* ocurrió este error, ni cómo describírselo apropiadamente al programador.
- Ambos aspectos son difíciles de implementar.

Localización del error

- Es complejo encontrar la ubicación exacta del token ofensivo.
- Y no necesariamente es un token mal puesto, el error puede ser más bien la *ausencia* de un token.

Localización del error

- Describir apropiadamente un error también es muy difícil.
- Peor de los casos “error de sintaxis” sin dar ningún otro detalle.



Recuperar

- Cuando se encuentra un error, ya no sirve la hilera, ni siquiera el programa.
- Pero si simplemente lo dejamos ahí... ¿Qué pasa si hay muchos errores? El programador tendría que solucionar el primer error y luego volver a utilizar el compilador para ver el siguiente. Sería muy impráctico.



Imagen: Persona recuperándose.

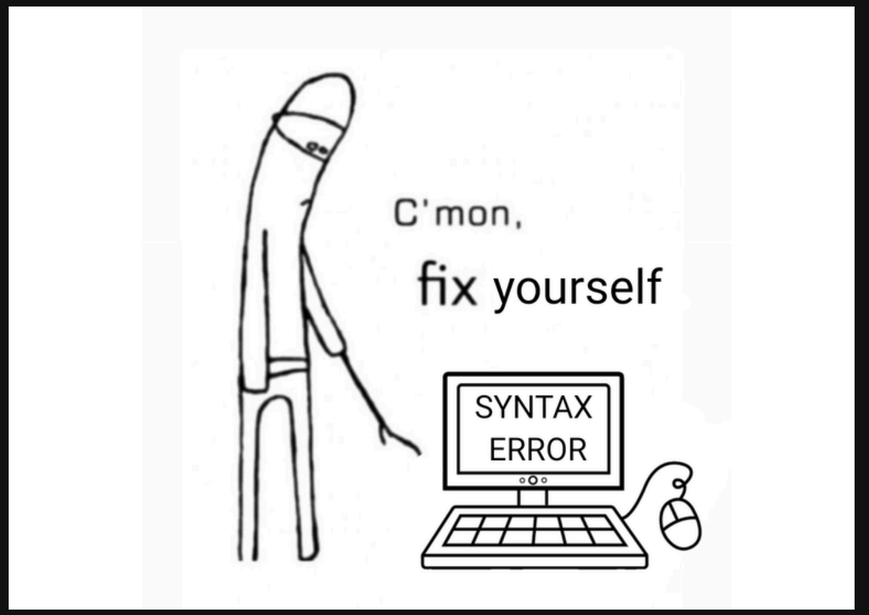
- Es mejor buscar todos los errores del programa de un solo.
- Por eso, el parser se debe recuperar luego de encontrar un error :).
- Así puede seguir analizando el resto del programa.
- Se debe elegir un “punto seguro” al que avanzar para continuar el proceso de parsing.
- ¿Cómo elegimos ese punto seguro? Si avanzamos mucho, podemos ignorar por accidente nuevos errores; si no avanzamos suficiente, nos encontramos el mismo error que ya fue detectado (cascada de errores).

Reparar

- Si el parser "sabe" qué está mal... ¿Por qué no lo soluciona?
- Hay veces que sí se hace esto como parte de la Recuperación. De hecho, los warnings pueden ser resultado de una reparación.
- Pero ¿realmente queremos que el compilador "arregle" todo nuestro código?
- La razón por la que es difícil formular un mensaje de error concreto es porque hay muchas explicaciones posibles para un error.
- Muchas explicaciones posibles implica muchas soluciones posibles.



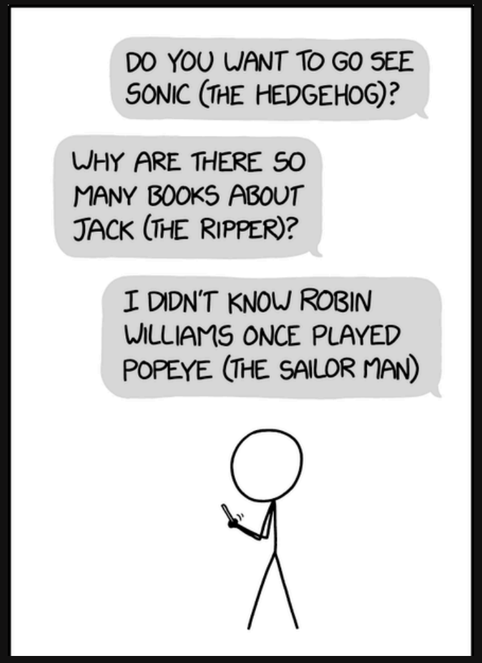




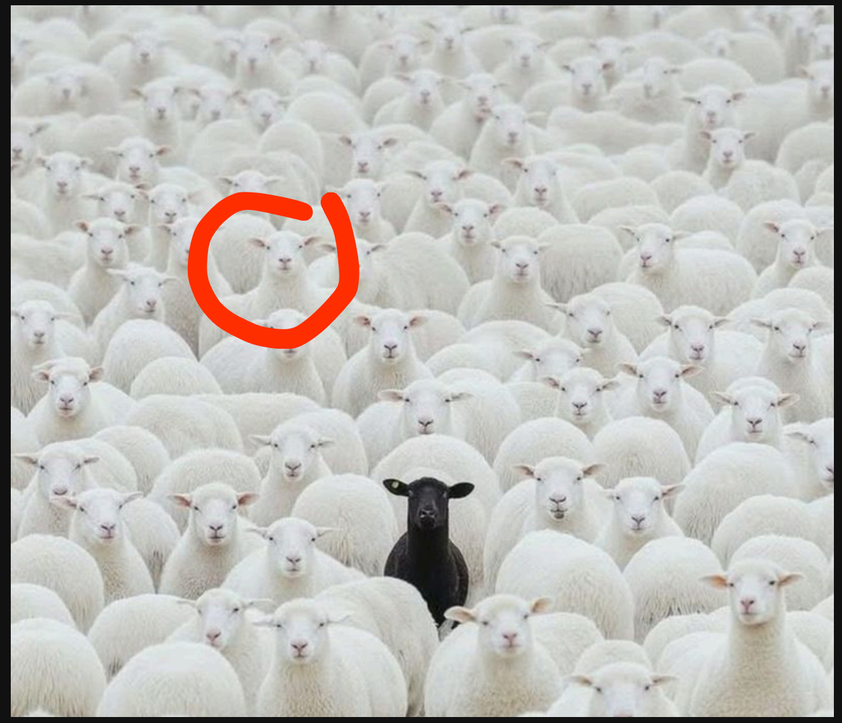
Traducción dirigida por sintaxis



- El parser dirige todo el proceso.
- El parser es el cliente del scanner, lo invoca cada vez que necesita un nuevo token.
- Se genera código intermedio que procesa el backend del compilador.
- Casi no hay compiladores que no usen traducción dirigida a sintaxis.







GRAMÁTICA LIBRE DE CONTEXTO

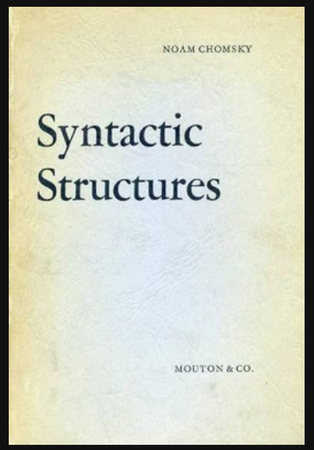


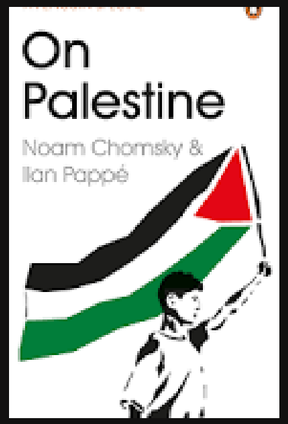
Avram Noam Chomsky



- Nació en 1928 y SIGUE VIVO!! [3]
- Conocido como el padre de la lingüística.
- Era un lingüista, filósofo y científico social estadounidense.
- Profesot emérito del M.I.T.
- Una de las figuras culturales más importantes de tiempos recientes.
- Es de los autores más citados.
- También es conocido por sus posiciones políticas anti-bélicas e izquierdistas.







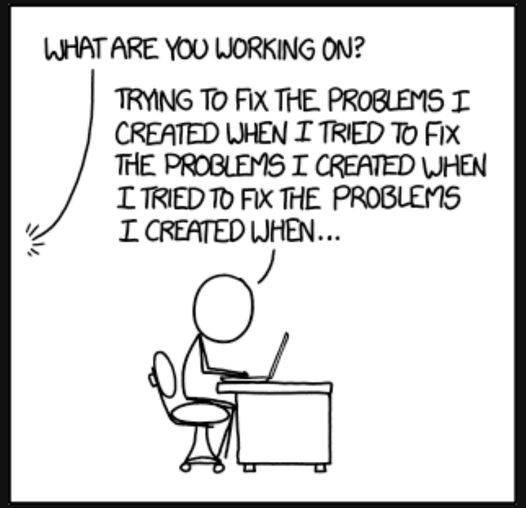


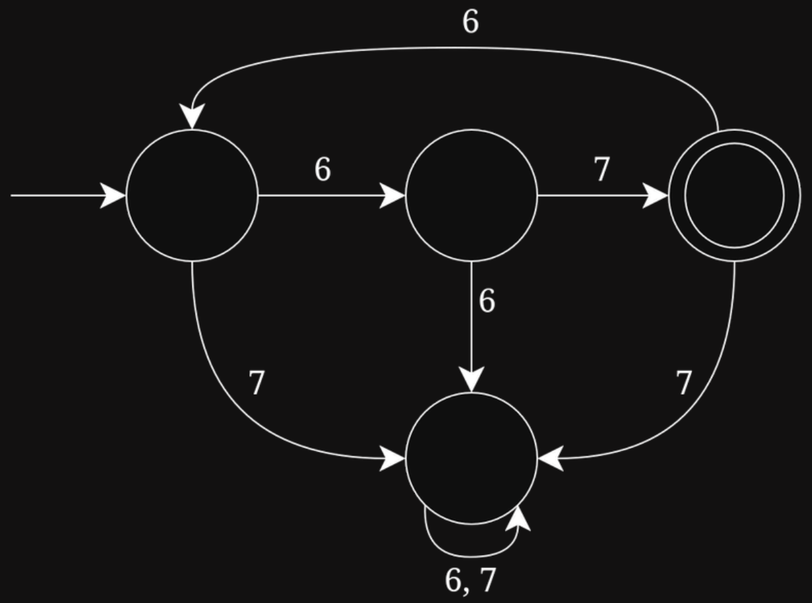


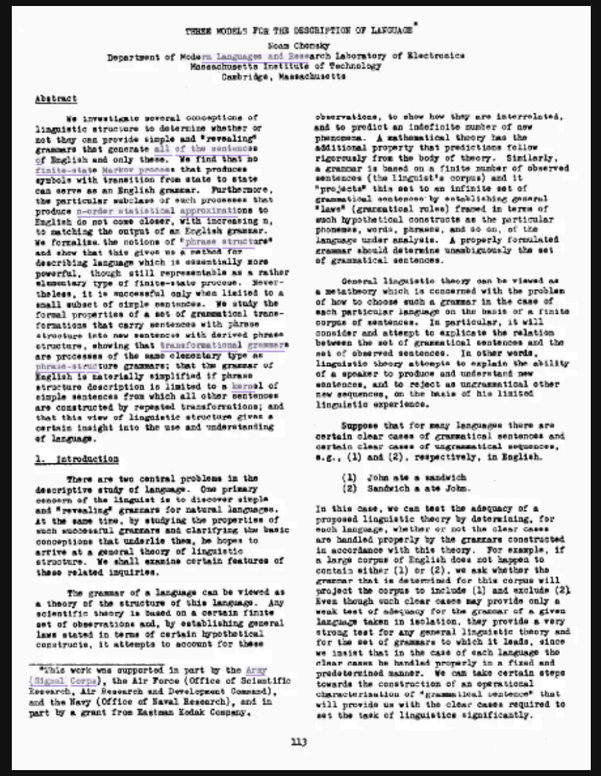
[3] Pregúntenle al profe Avilés.

Lenguajes No Regulares

- El pumping lemma nos permitió demostrar que existen lenguajes no regulares.
  - Por ejemplo: el lenguaje L sobre Σ = {a,b} definido como $\{a^n b^n \mid n \leq 0\}$.
- ¿Podrían ser generados y reconocidos por algún mecanismo?
- La Jerarquía de Chomsky es una clasificación jerárquica de distintos tipos de gramáticas formales que generan lenguajes formales. Esta jerarquía fue descrita por Chomsky en 1956. [4]







THREE MODELS FOR THE DESCRIPTION OF LANGUAGE*

Noam Chomsky
Department of Modern Languages and Research Laboratory of Electronics
Massachusetts Institute of Technology
Cambridge, Massachusetts

Abstract

We investigate several conceptions of linguistic structure to determine whether or not they can provide simple and "revealing" grammars that generate all of the sentences of English and only these. We find that no finite-state Markov process that produces symbols with transition from state to state can serve as an English grammar. Furthermore, the particular subclass of such processes that produce n-order statistical approximations to English do not come closer, with increasing n, to matching the output of an English grammar. We formalize the notions of "phrase structure" and show that this gives us a method for describing language which is essentially more powerful, though still representable as a rather elementary type of finite-state process. Nevertheless, it is successful only when limited to a small subset of simple sentences. We study the formal properties of a set of grammatical transformations that carry sentences with phrase structure into new sentences with derived phrase structure, showing that transformational grammars are processes of the same elementary type as phrase-structure grammars; that the grammar of English is materially simplified if phrase structure description is limited to a kernel of simple sentences from which all other sentences are constructed by repeated transformations; and that this view of linguistic structure gives a certain insight into the use and understanding of language.

1. Introduction

There are two central problems in the descriptive study of language. One primary concern of the linguist is to discover simple and "revealing" grammars for natural languages. At the same time, by studying the properties of such successful grammars and clarifying the basic conceptions that underlie them, he hopes to arrive at a general theory of linguistic structure. We shall examine certain features of these related inquiries.

The grammar of a language can be viewed as a theory of the structure of this language. Any scientific theory is based on a certain finite set of observations and, by establishing general laws stated in terms of certain hypothetical constructs, it attempts to account for these observations, to show how they are interrelated, and to predict an indefinite number of new phenomena. A mathematical theory has the additional property that predictions follow rigorously from the body of theory. Similarly, a grammar is based on a finite number of observed sentences (the linguist's corpus) and it "projects" this set to an infinite set of grammatical sentences by establishing general "laws" (grammatical rules) framed in terms of such hypothetical constructs as the particular phonemes, words, phrases, and so on, of the language under analysis. A properly formulated grammar should determine unambiguously the set of grammatical sentences.

General linguistic theory can be viewed as a metatheory which is concerned with the problem of how to choose such a grammar in the case of each particular language on the basis of a finite corpus of sentences. In particular, it will consider and attempt to explicate the relation between the set of grammatical sentences and the set of observed sentences. In other words, linguistic theory attempts to explain the ability of a speaker to produce and understand new sentences, and to reject as ungrammatical other new sequences, on the basis of his limited linguistic experience.

Suppose that for many languages there are certain clear cases of grammatical sentences and certain clear cases of ungrammatical sequences, e.g., (1) and (2), respectively, in English.

(1) John ate a sandwich
(2) Sandwich a ate John.

In this case, we can test the adequacy of a proposed linguistic theory by determining, for each language, whether or not the clear cases are handled properly by the grammars constructed in accordance with this theory. For example, if a large corpus of English does not happen to contain either (1) or (2), we ask whether the grammar that is determined for this corpus will project the corpus to include (1) and exclude (2). Even though such clear cases may provide only a weak test of adequacy for the grammar of a given language taken in isolation, they provide a very strong test for any general linguistic theory and for the set of grammars to which it leads, since we insist that in the case of each language the clear cases be handled properly in a fixed and predetermined manner. We can take certain steps towards the construction of an operational characterization of "grammatical sentence" that will provide us with the clear cases required to set the task of linguistics significantly.

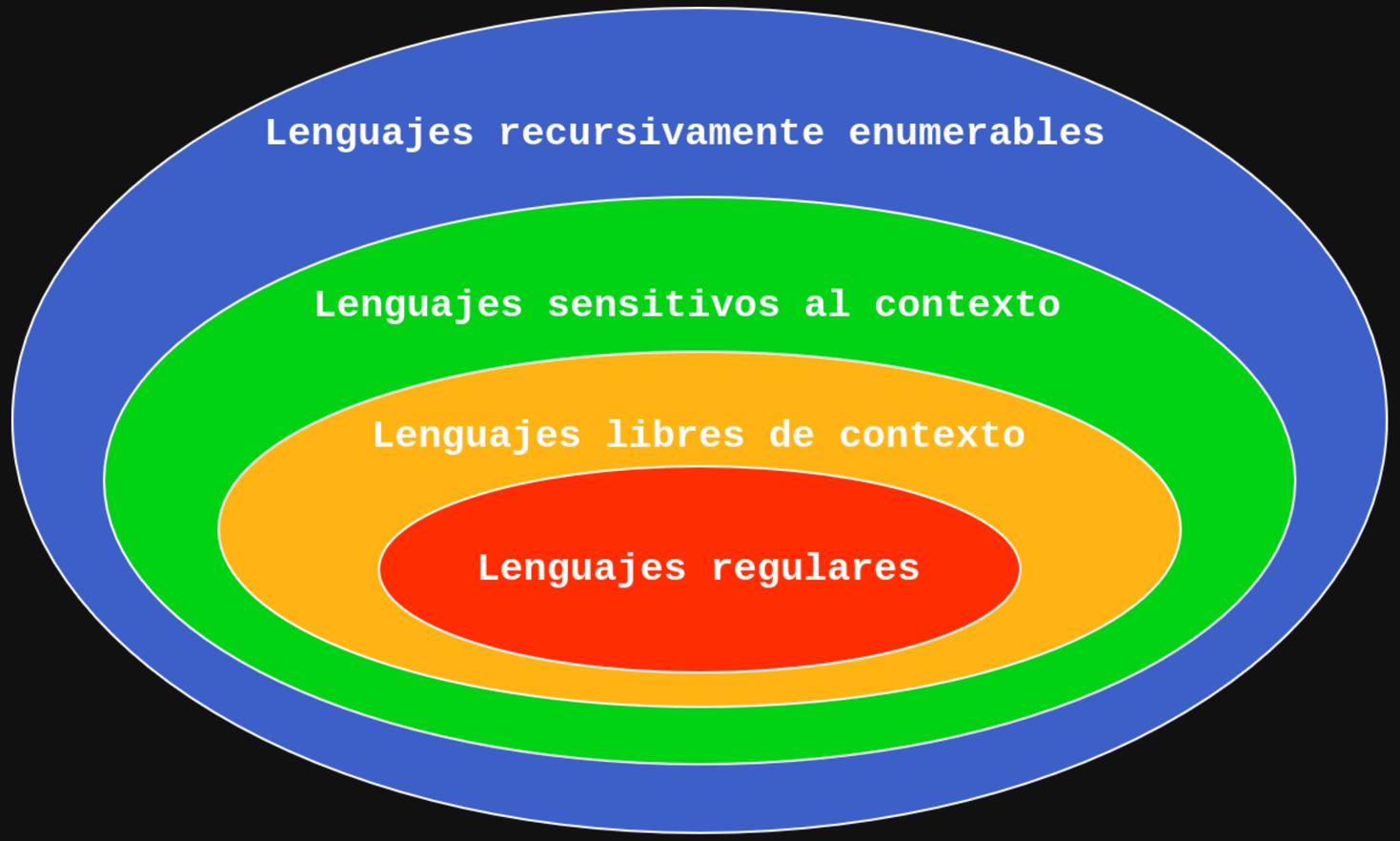
*This work was supported in part by the Army (Signal Corps), the Air Force (Office of Scientific Research, Air Research and Development Command), and the Navy (Office of Naval Research), and in part by a grant from Eastman Kodak Company.

113

[4] Saqué la descripción de Wikipedia :p. Link.

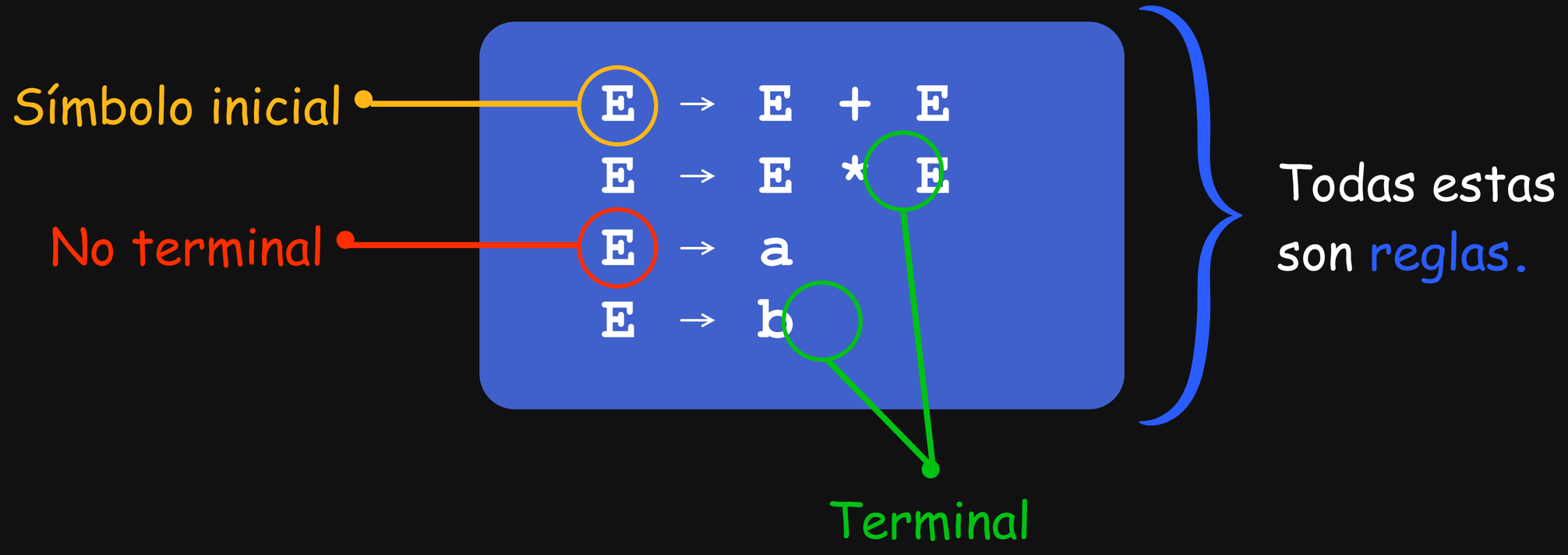
Jerarquía de Chomsky

Los lenguajes no regulares calzan en los niveles luego de "lenguajes regulares". Solo no pertenecen en ese primero.



Gramática libre de conexto

Esta es una gramática libre de contexto y sus componentes:



Nunca vamos a ver terminales en el lado izquierdo de una gramática.

Definición formal



Una Gramática Libre de Contexto (CFG) G es un cuarteto (V, Σ, P, S) tal que:

- V es un conjunto finito de no terminales
- Σ es un conjunto finito de terminales
- P es un conjunto finito de reglas
- $S \in V$ es el símbolo inicial

Un par de aclaraciones también:

- $V \cap \Sigma = \varnothing$. Es decir, los terminales y no terminales son conjuntos disjuntos. No se puede ser terminal y no terminal al mismo tiempo. Decídanse :/.
- Las relgas tienen la forma $P \subseteq V \times (V \cup \Sigma)^*$. Es decir, tienen un único no terminal a la izquierda y una hilera de no terminales a la derecha. Esta hilera puede ser ε.





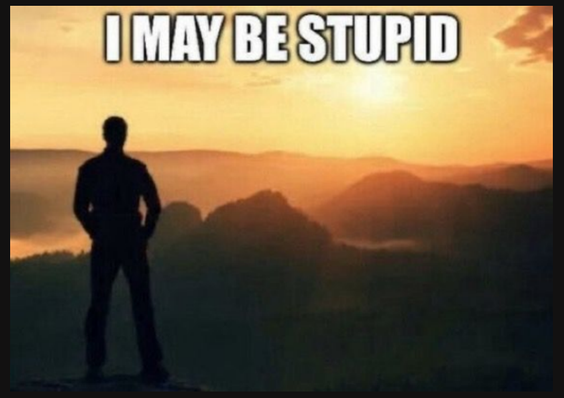
Derivación

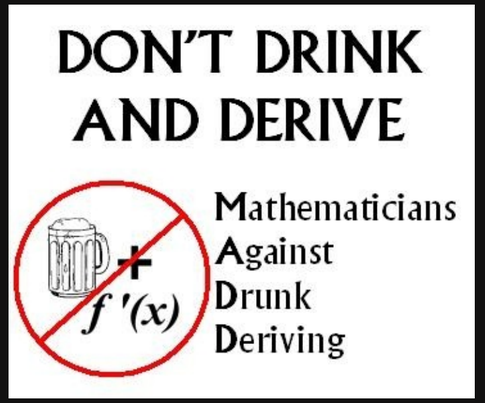
Las CFG se "expanden" derivando.
Sean α, β, γ hileras tomadas de (V ∪ Σ)* (las hileras que aparecen en las reglas):

- Si se tienen que una hilera w de la forma αAB y hay una regla en la gramática de la forma A → γ entonces w se puede reescribir como αγβ.
- Decimos que w deriva a la hilera αγβ se denota como w => αγβ
- Se puede derivar mientras hayan no terminales.
- Para aplcicar una regla solo nos interesa la presencia dle No Terminal que se va a expandir. El contexto que lo rodea no importa. [5]

Notación de Derivación

- Un paso de derivación w => v
- Uno o más pasos de derivación w =>$^{+}$v
- Cero o más pasos de derivación: w => *v





[5] Es casi como si fueran.... libres de contexto.... hm.

Algoritmo de derivación

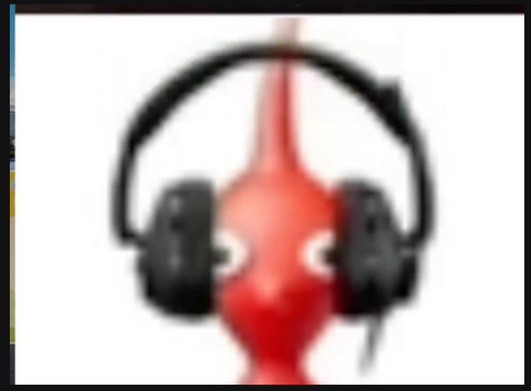
- Sea $G = (V, \Sigma, P, S)$ una CFG
- Sea $w \in (V \cup \Sigma)^*$

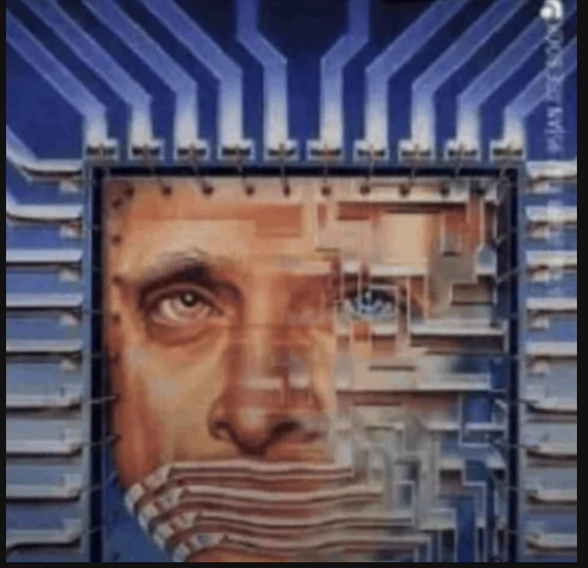
El conjunto de hileras derivables de w se define recursivamente como:

1. w es derivable de w.
2. Si $v = \alpha A \beta$ es derivable desde w y $A \rightarrow \gamma \in P$ entonces $\alpha\gamma\beta$ es derivable desde w.
3. Todas y únicamente las hileras construidas aplicando el paso 2 son derivables desde w.

Hileras derivables desde w = { u | w => *v }









Ejemplo

Sea G definida como:

$$E \rightarrow E + E$$
$$E \rightarrow E * E$$
$$E \rightarrow a$$
$$E \rightarrow b$$

¿Hileras derivables de “abba”?

R\ ??????

¿Hileras derivables de *EaE?

R\ ??????????????



Ejemplo

Sea G definida como:

$$E \rightarrow E + E$$
$$E \rightarrow E * E$$
$$E \rightarrow a$$
$$E \rightarrow b$$

¿Hileras derivables de “abba”?

R\ {abba}

¿Hileras derivables de *EaE?

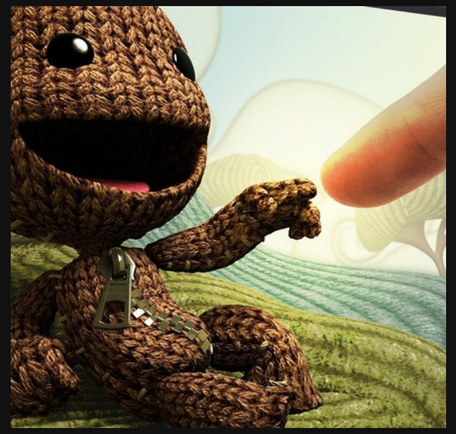
R\ {*EaE, *aaE, *aaa, E+EaE, *E*EaE, ...}

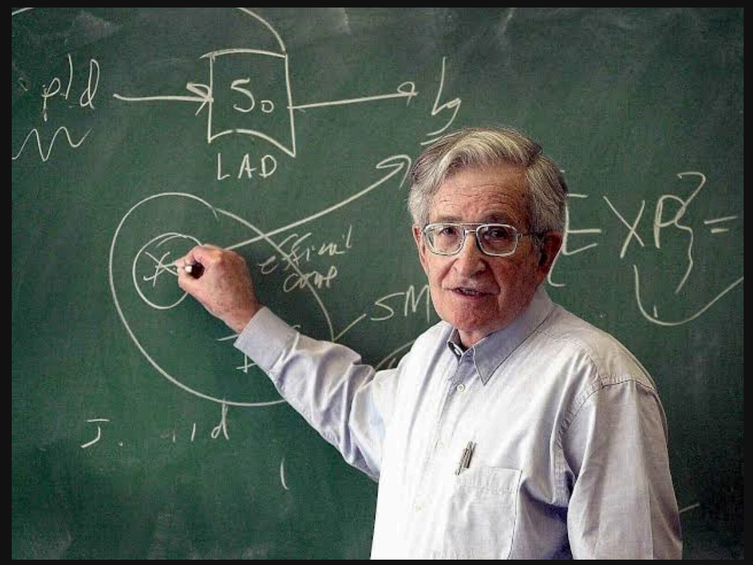


Definiciones

Sea G = (V, Σ, P, S) una CFG

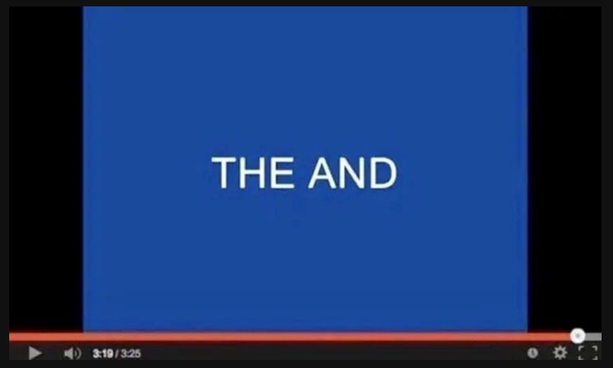
1. Una hilera w ∈ (V ∪ Σ)* es una forma sentencial de G si S => *w
2. Una hilera w ∈ Σ* es una forma sentencial de G si S => *1
3. El lenguaje de G, denotado como L(G) es el cinjunto { v | v ∈ Σ* y S => *v }
4. El lenguaje de G es el conjunto de todas las sentencias de G.







En los próximos apuntes (15/10/26) podrán ver muchos ejemplos!!



GRACIAS POR LEER :)

Ya no tengo más memes. espero disfruten mucho este slide.